# Adaptive Atmospheric Turbulence Restoration for Remote Sensing Images with Generative Models

**Sitian Ding** 2025011222
**Jiamu Qin** 2025011005
**Wenyuan Huang** 2025011051

## Abstract

Atmospheric turbulence introduces spatially varying blur, geometric warping, and temporal instability that significantly degrade long-range remote sensing imagery. Unlike standard image corruption, turbulence is non-uniform across space and time, which makes direct restoration difficult and also makes paired real-world supervision scarce. In this project, we formulate adaptive atmospheric turbulence restoration for remote sensing as a supervised image-to-image translation problem driven by physically simulated degradations. We build a compact training pipeline on top of UC Merced Land Use and NWPU-RESISC45 imagery, synthesize turbulence-corrupted seven-frame sequences with a TurbulenceSim-based simulator, and train a multi-frame U-Net baseline that stacks temporal observations along the channel dimension. On the validation split, the resulting model reaches 35.34 dB PSNR and 0.9804 SSIM, while qualitative results show improved recovery of roads, parking lots, buildings, and man-made boundaries under strong blur and local distortion. Code and assets are available at `https://github.com/Mars-Dingdang/DL-AdaptiveOptics`, and the dataset release is available at `https://huggingface.co/datasets/Mars-Dingdang/DL-AdaptiveOptics`.

## 1 Introduction

Atmospheric turbulence is a major challenge for long-range imaging, surveillance, astronomy, and remote sensing. Temperature and pressure fluctuations cause random variations in the refractive index along the optical path, which in turn induce geometric dancing, spatially varying blur, and local intensity fluctuations. In imagery, the most visible artifacts are wavy edges, unstable fine structures, and locally inconsistent sharpness. A recent review by Hill et al. emphasizes that turbulence is especially difficult because the distortion is spatially varying and spatio-temporal rather than globally shift-invariant or static (Hill et al., 2024). These effects reduce human interpretability and degrade downstream tasks such as scene classification, land-use analysis, and object recognition.

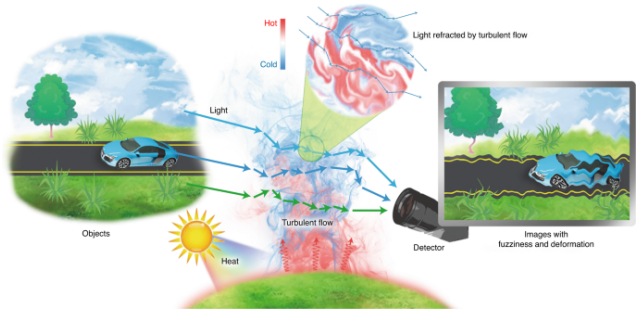



Figure 1: Schematic of imaging through atmospheric turbulence, extracted from Jin et al. (2021).

Preprint.

For remote sensing, the restoration problem is even harder than for common natural images. First, aerial and satellite scenes often contain repeated textures, man-made lines, and small semantic targets whose visibility depends on high-frequency spatial detail. Second, collecting paired data with both turbulence-corrupted observations and perfectly aligned clean ground truth is usually impractical. Third, turbulence in real imaging systems interacts with viewing distance, aperture, focal length, wavelength, wind speed, and scene depth, so synthetic approximations must balance fidelity and throughput.

Existing restoration methods fall into two broad families. Adaptive optics methods physically correct the distorted wavefront using specialized hardware, but they are expensive and not always practical for remote sensing acquisition pipelines. Image processing and learning-based methods instead aim to invert the corruption in software. Earlier classical approaches relied on registration, lucky-region fusion, deconvolution, or low-rank priors, while recent deep models learn the restoration mapping directly from data. TSR-WGAN showed that spatio-temporal generative modeling can recover complex scenes from turbulence sequences (Jin et al., 2021). LTT-GAN further demonstrated that adversarial learning and latent inversion can restore images through turbulence more effectively than purely hand-crafted methods (Mei and Patel, 2023). Diffusion-based restoration has recently become attractive because it can model highly multimodal inverse problems and incorporate physical priors or zero-shot constraints (Wang et al., 2023; Jaiswal et al., 2023).

However, most turbulence restoration literature targets surveillance or generic natural images. Remote sensing has different texture statistics, larger viewpoint variation, and stronger dependence on structural fidelity. This project therefore focuses on a practical remote sensing setting and asks a narrower question: can a relatively compact, physically grounded pipeline already provide strong turbulence restoration on remote sensing patches using limited training data? Our answer is to combine public aerial scene datasets with a TurbulenceSim-style degradation model and train a multi-frame U-Net that exploits temporal redundancy across seven turbulence-corrupted observations. In this formulation, the task becomes learning a mapping from a degraded sequence to a clean reference image, which preserves the physical motivation of adaptive optics while remaining tractable as a deep learning course project.

## 2 Method

Our pipeline consists of three stages: selecting clean remote sensing imagery, generating paired turbulence-corrupted sequences with a physical simulator, and training a supervised restoration network on those synthetic pairs.

### 2.1 Data selection

We use two public remote sensing datasets as clean image sources: UC Merced Land Use and NWPU-RESISC45. UC Merced offers diverse aerial scenes such as agricultural areas, runways, harbors, parking lots, and residential blocks. NWPU-RESISC45 provides broader scene diversity and larger scale coverage, which makes it better suited for extracting a large number of training patches. In the current repository configuration, clean image patches are extracted from NWPU-RESISC45 and used to build the turbulence sequence dataset, while UC Merced remains part of the overall project data pool and supports broader discussion of remote sensing scene types.

To keep the problem computationally manageable, we train on only 2000 synthesized samples. Each sample corresponds to one clean target image patch and one seven-frame turbulence-degraded observation sequence. Images are cropped or resized to $256 \times 256$. The configuration sets a validation split ratio of 0.1, which yields a compact but sufficient held-out subset for model selection.

### 2.2 Physical degradation model

Paired real-world turbulence data for remote sensing are difficult to collect because one would need both a turbulence-corrupted observation and a perfectly aligned turbulence-free target. We therefore rely on simulation. The project uses a TurbulenceSim-based degradation pipeline, motivated by the propagation-free simulator of Chimitt and Chan (Chimitt and Chan, 2020), which is designed to balance physical interpretability, anisoplanatic realism, and practical speed.

### 2.2.1 Simple Parametric Degradation

The implementation in this repository uses a lightweight fallback backend rather than a full physical turbulence model. In the code, the `simple_parametric` path applies a random subpixel translation, a Gaussian blur whose strength depends on the sampled context, Poisson noise, Gaussian read noise, and JPEG compression. For sequence synthesis, each frame is generated independently with fresh jitter and noise, so the degraded sequence exhibits temporal variation without explicitly simulating an anisoplanatic wavefront.

More explicitly, let $x$ denote the clean image and let $W_{\Delta_t}$ be a random translation warp sampled from a zero-mean Gaussian displacement $\Delta_t = (\delta x_t, \delta y_t)$. The warped image is then blurred by a Gaussian kernel with frame-dependent standard deviation $\sigma_t$, giving

$$\tilde{x}_t = G_{\sigma_t}\big(W_{\Delta_t}(x)\big), \tag{1}$$

where $G_{\sigma_t}$ denotes channel-wise Gaussian blur. The code then applies sensor noise by first drawing Poisson counts with a sampled scale $s_t$ and then adding Gaussian read noise,

$$\hat{x}_t = \frac{1}{s_t}\mathrm{Poisson}(s_t \tilde{x}_t) + n_t^{(G)}, \qquad n_t^{(G)} \sim \mathcal{N}(0, \sigma_{n,t}^2), \tag{2}$$

and finally JPEG compression with quality factor $q_t$ is applied,

$$y_t = \mathcal{J}_{q_t}(\hat{x}_t). \tag{3}$$

In the implementation, the sampled context controls the translation and blur scale through wind speed, $C_n^2$, and the turbulence-strength knob, while the noise and JPEG settings are drawn from fixed ranges. This is best viewed as a fast synthetic corruption model, not as an explicit phase-screen simulator.

### 2.2.2 TurbulenceSim Degradation

The repository wraps the upstream TurbulenceSim-v1 implementation through two adapters: a CPU-oriented backend and a GPU-oriented backend. The configuration in the repository defaults to the CPU-capable adapter in code. The parameters exposed by the project include Zernike order 6, phase strength 0.12, PSF kernel size 5, turbulence strength 0.50 in the default YAML, refractive-index structure parameter range $C_n^2 \in [10^{-16}, 5 \times 10^{-15}]$, focal length range 6000–18000, wind speed range 0.5–2.0, aperture diameter 0.2 m, wavelength $5.25 \times 10^{-7}$ m, object size 2.06, and frame time step 0.03 s. The GPU adapter additionally exposes luminance-only simulation, patch-grid downsampling, PSF resolution control, and per-frame PSF reuse flags.

Following Chimitt and Chan, the upstream simulator first derives the Fried coherence diameter

$$r_0 = \left(0.423 k^2 C_n^2 L\right)^{-3/5}, \qquad k = \frac{2\pi}{\lambda}, \tag{4}$$

where $L$ is the propagation distance and $\lambda$ is the wavelength. In the adapters, $r_0$ is clipped to a stable range and then used to build the upstream TurbulenceSim parameter object. The CPU path calls `p_obj`, precomputes the tilt PSD with `gen_PSD`, applies random local motion with `genTiltImg`, and then synthesizes spatially varying blur with `genBlurImage`. The blur stage uses patch-wise Zernike draws inside the upstream code rather than a single global PSF. A frame-level sinusoidal modulation of $r_0$ is also used so that the sequence changes smoothly over time. The most compact mathematical summary is that the upstream code generates a random local phase field, converts it to a PSF, and applies it patch by patch.

At the level of Zernike modeling, the turbulence phase at a pupil location can be written as

$$\phi(\rho, \theta) = \sum_{j=1}^{J} a_j Z_j(\rho, \theta), \tag{5}$$

where the random coefficients $a_j$ are sampled jointly rather than independently. The paper represents their spatial and inter-modal coupling with a covariance matrix

$$C_{j,j'} = \mathbb{E}[a_j^* a_{j'}], \qquad \mathbf{a} \sim \mathcal{N}(\mathbf{0}, C), \tag{6}$$

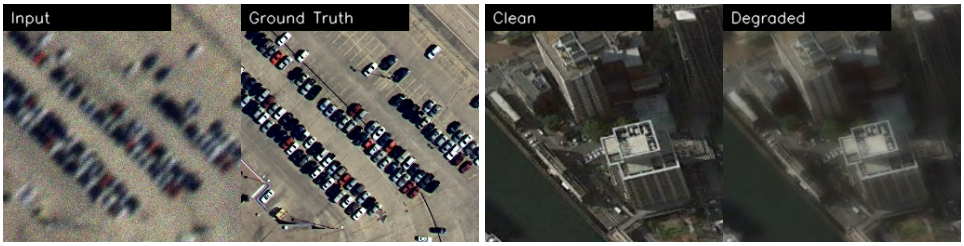
which is the key device used to generate spatially correlated Zernike draws across the image. Following the classical result cited in the paper, the 2nd and 3rd Zernike coefficients mainly account for tilt, while the higher-order coefficients mainly contribute to blur.

The GPU adapter reuses the same upstream operators, but it can switch to luminance-only processing, downsample the patch grid, reuse one PSF per frame, and change the PSF resolution. The final output is blended back with the clean image using a strength gate clipped to the interval $[0, 1]$.

This is the main difference from the simple parametric backend: the TurbulenceSim path delegates spatially varying motion and patch-wise PSF generation to the upstream implementation, so it captures anisoplanatic blur and warp much more faithfully than a single global corruption kernel.

### 2.2.3 Visualized Comparison

In practice, we found that the simple parametric backend using Zernike coefficients, motion blur, Poisson and Gaussian noise, degraded the images only with intensive noise points but didn't simulate the spatially varying blur of real turbulence. The default CPU TurbulenceSim backend gave a much more realistic degradation, as is shown in Figure 2.




(a) Simple Parametric Degradation (b) Default TurbulenceSim Degradation

Figure 2: Comparison of two degradation pipelines.

Among different settings of TurbulenceSim parameters (especially turbulence strength), the default configuration with strength 0.5 already produces blurring and local distortion visually similar to real atmospheric turbulence. Increasing the strength to 1.0 eventually leads to severe blur and turbulence. The visualized 7-frame images with turbulence strength ranging from 0.5 to 1.0 are shown in Figure 3.

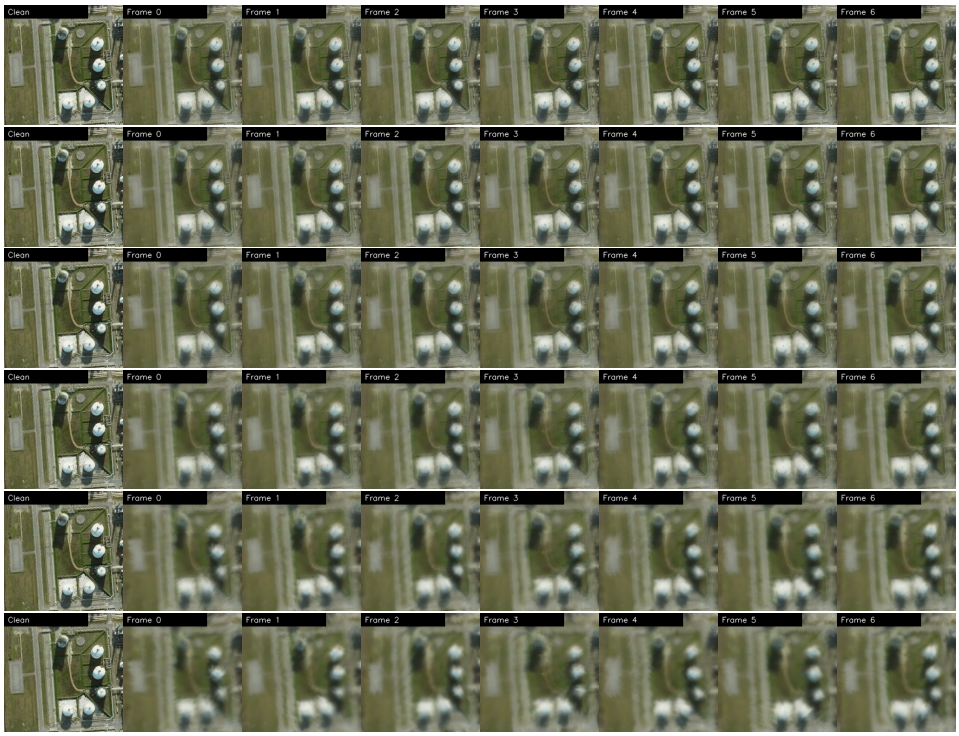



Figure 3: Visualized 7-frame images with turbulence strength ranging from 0.5 to 1.0.

## 2.3 Model Architecture

### 2.3.1 Multi-frame U-Net architecture baseline

The restoration backbone is a baseline U-Net defined in the repository. It uses a standard encoder-decoder structure with four downsampling stages, four upsampling stages, skip connections, batch normalization, and ReLU activations. The base channel width is 64. Since the task is supervised image restoration, the network predicts a clean RGB image and is trained with an $L_1$ reconstruction loss.

The distinctive design choice is the input representation. Instead of restoring from a single frame, we use seven degraded frames for each sample. The training code supports several sequence-to-image adaptation strategies, and the active configuration uses channel stacking. Concretely, a sequence tensor of shape $B \times 7 \times 3 \times H \times W$ is reshaped into $B \times 21 \times H \times W$ before entering the network. This design is motivated by the observation that turbulence has temporal variation but also local periodicity or recurrence in distortion patterns. Multiple frames provide complementary glimpses of the clean scene: regions severely blurred in one frame may be less degraded in another. The 7-frame U-Net therefore receives a richer observation than a single-frame baseline while remaining simple enough to train stably.

With 21 input channels, 3 output channels, and base width 64, the network contains approximately 31.0 million trainable parameters. Although this is still a standard architecture rather than a specialized transformer or diffusion model, it provides a useful baseline for understanding how much performance can be extracted from physical simulation plus temporal stacking alone.

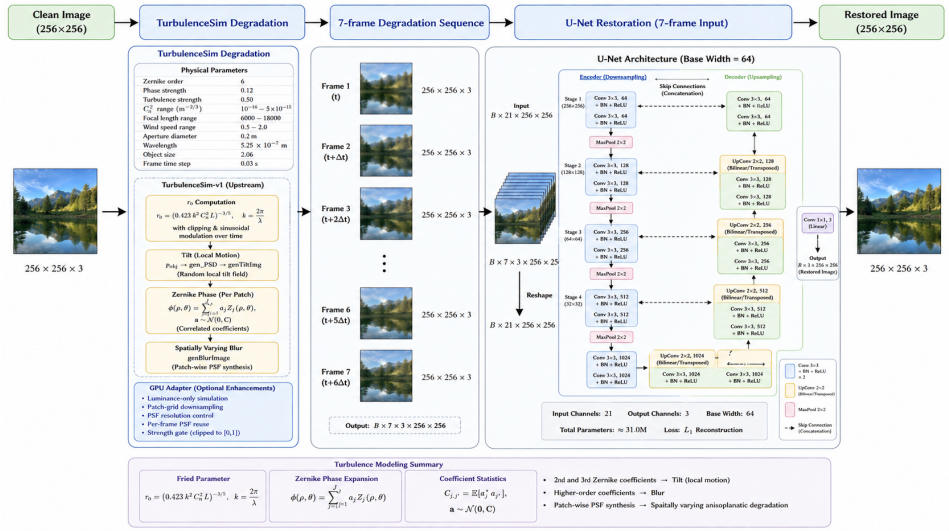



Figure 4: Schematic of the multi-frame U-Net architecture and data pipeline (GPT-image-2 generated).

## 2.4 Objectives and training details

### 2.4.1 Metrics and objective

Training follows the implementation in `train_unet.py`: the model predicts a restored RGB image, the prediction is clamped to $[0, 1]$ before loss evaluation, and the optimization objective is the mean $L_1$ reconstruction loss against the clean target. For a minibatch of size $B$, with prediction $\hat{y}_i$ and clean target $y_i$, the loss used in training is

$$\mathcal{L}_{L_1} = \frac{1}{B} \sum_{i=1}^{B} \frac{1}{CHW} \sum_{c=1}^{C} \sum_{h=1}^{H} \sum_{w=1}^{W} \left| \mathrm{clip}(\hat{y}_i)_{c,h,w} - y_{i,c,h,w} \right|, \tag{7}$$

where $\mathrm{clip}(\cdot)$ truncates each predicted pixel to the interval $[0, 1]$. During validation, the code computes batch-level PSNR and SSIM on the clamped predictions in the same $[0, 1]$ range, using the

full RGB image with channel-aware SSIM from `skimage`. The resulting metrics are averaged over the validation set, and checkpoint selection is based on validation PSNR. An optional LPIPS term is also implemented in the metric wrapper, but it is disabled in the default configuration.

PSNR provides a simple error-based quality score. With images normalized to $[0, 1]$, it is computed from the mean squared error as

$$\mathrm{MSE}(y, \hat{y}) = \frac{1}{CHW} \sum_{c=1}^{C} \sum_{h=1}^{H} \sum_{w=1}^{W} (y_{c,h,w} - \hat{y}_{c,h,w})^2, \qquad \mathrm{PSNR}(y, \hat{y}) = 10 \log_{10} \left( \frac{1}{\mathrm{MSE}(y, \hat{y})} \right), \tag{8}$$

so higher values indicate smaller pixel-wise distortion. SSIM, following Wang et al. (Wang et al., 2004), is designed to better reflect perceptual structure by comparing local luminance, contrast, and structural consistency rather than only raw error. For two local patches $x$ and $y$, the classic SSIM form is

$$\mathrm{SSIM}(x, y) = \frac{(2\mu_x \mu_y + c_1)(2\sigma_{xy} + c_2)}{(\mu_x^2 + \mu_y^2 + c_1)(\sigma_x^2 + \sigma_y^2 + c_2)}, \tag{9}$$

where $\mu_x$ and $\mu_y$ are local means, $\sigma_x^2$ and $\sigma_y^2$ are local variances, $\sigma_{xy}$ is local covariance, and $c_1, c_2$ are stabilizing constants. In this report, PSNR is used as the primary selection metric, while SSIM is reported to summarize structural fidelity.

#### 2.4.2 Training details

Training is launched through the unified script but dispatched to a dedicated U-Net entry point. The optimizer is Adam with learning rate $2 \times 10^{-4}$, $\beta_1 = 0.5$, $\beta_2 = 0.999$, and zero weight decay. A StepLR scheduler halves the learning rate every 30 epochs. The model is trained for 50 epochs using automatic mixed precision when CUDA is available. The per-process training batch size is 64 and validation batch size is 8.

The code also supports distributed data parallel training through torchrun. When launched with multiple GPUs, the project initializes an NCCL process group, assigns one GPU per process, uses distributed samplers, and aggregates metrics across ranks. This is especially relevant for the current setting because sequence inputs increase memory pressure compared with single-frame restoration. In our configuration, the loss is plain $L_1$ between the restored image and the clean target, and checkpoint selection is based on validation PSNR.

## 3 Results and Analysis

### 3.1 Quantitative results

#### 3.1.1 Validation performance

The best U-Net checkpoint evaluated on the validation split reaches the metrics shown in Table 1. The reported values come from the repository output generated by the evaluation pipeline.

Table 1: Validation performance of the 1-frame/7-frame U-Net restoration model.

| Model | Input setting | Data turbulence strength | PSNR ↑ | SSIM ↑ |
|---|---|---|---|---|
| U-Net baseline | 7 frames stacked | 0.5 | 35.341 | 0.9804 |
| U-Net baseline | 1 frame | 0.5 | 31.483 | 0.9581 |
| U-Net baseline | 7 frames stacked | 0.75 | 31.783 | 0.9557 |
| U-Net baseline | 1 frame | 0.75 | 29.463 | 0.9256 |

The absolute values indicate that the model recovers a large portion of the structural signal from the simulated degradation. A PSNR above 35 dB on $256 \times 256$ remote sensing patches suggests that the network is not merely smoothing the image, but is reconstructing scene content in a way that remains close to the clean target. The SSIM value of 0.9804 further supports that most structural layout and local contrast relationships are preserved.

At the same time, these numbers should be interpreted carefully. The evaluation uses synthetic degradations drawn from the same simulator family as training, so the results primarily measure how well the network learns to invert the chosen physical data model. They do not yet guarantee the same level of performance on real atmospheric turbulence data. This gap between simulated and real degradation remains one of the key limitations of turbulence restoration research. We will dive deeper into this issue in the following parts.

### 3.1.2 Generalization across input settings and turbulence strengths

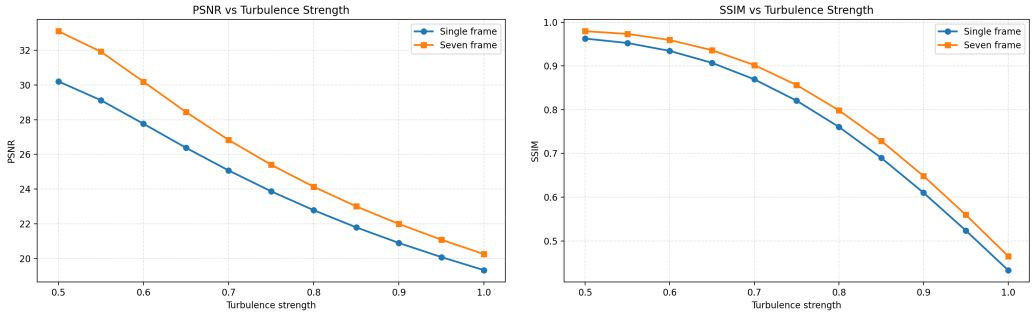
We also studied the generalization ability of the baseline U-Net model trained on 7-frame input of 0.5 turbulence strength by testing it on both single-frame and seven-frame inputs with different turbulence strength ranging from 0.5 to 1.0. The metrics are shown in Table 2 and Figure 5.

The results show that the 7-frame U-Net model trained on 7-frame input can generalize to single-frame input only with a performance drop of approximately 2 dB in PSNR and 0.02 in SSIM across different turbulence strengths. This suggests that the model has learned some robust features that are not entirely dependent on the multi-frame input, but the performance gap also indicates that the temporal information from multiple frames is valuable for achieving higher restoration quality.

However, the performance drops significantly as turbulence strength increases both for single-frame and multi-frame inputs, which is expected since stronger turbulence degradation leaves less information available and is more challenging to restore.

Table 2: Generalization performance of the 7-frame U-Net model trained on 0.50 turbulence strength across different turbulence strengths and input settings.

| Turbu-strength | PSNR (1 frame) ↑ | SSIM (1 frame) ↑ | PSNR (7 frames) ↑ | SSIM (7 frames) ↑ |
|---|---|---|---|---|
| 0.5 | **30.21** | **0.9626** | **33.10** | **0.9794** |
| 0.55 | 29.12 | 0.9525 | 31.92 | 0.9733 |
| 0.6 | 27.76 | 0.9344 | 30.19 | 0.9593 |
| 0.65 | 26.38 | 0.9071 | 28.44 | 0.9360 |
| 0.7 | 25.07 | 0.8694 | 26.83 | 0.9020 |
| 0.75 | 23.87 | 0.8206 | 25.40 | 0.8564 |
| 0.8 | 22.78 | 0.7605 | 24.13 | 0.7985 |
| 0.85 | 21.79 | 0.6898 | 23.00 | 0.7287 |
| 0.9 | 20.89 | 0.6101 | 21.99 | 0.6483 |
| 0.95 | 20.08 | 0.5237 | 21.08 | 0.5595 |
| 1.0 | 19.32 | 0.4330 | 20.25 | 0.4651 |




(a) PSNR across turbulence strengths.

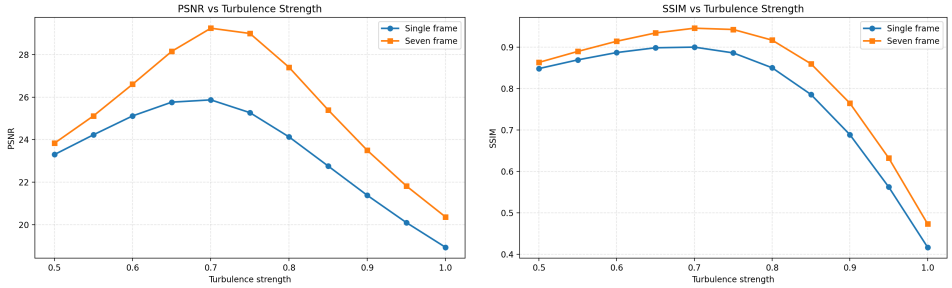
(b) SSIM across turbulence strengths.

Figure 5: Generalization performance of the 7-frame U-Net model trained on 0.50 turbulence strength across different turbulence strengths and input settings.

However, when we studied the generalization performance of the 7-frame U-Net model trained on 0.75 turbulence strength, the metrics are not monotonically decreasing as turbulence strength increases. The metrics are shown in Table 3 and Figure 6.

Table 3: Generalization performance of the 7-frame U-Net model trained on 0.75 turbulence strength across different turbulence strengths and input settings.

| Turbu-strength | PSNR (1 frame) ↑ | SSIM (1 frame) ↑ | PSNR (7 frames) ↑ | SSIM (7 frames) ↑ |
|---|---|---|---|---|
| 0.5 | 23.30 | 0.8485 | 23.83 | 0.8634 |
| 0.55 | 24.23 | 0.8695 | 25.12 | 0.8899 |
| 0.6 | 25.12 | 0.8872 | 26.60 | 0.9145 |
| 0.65 | 25.76 | 0.8988 | 28.15 | 0.9345 |
| 0.7 | **25.87** | **0.9003** | **29.24** | **0.9460** |
| 0.75 | 25.27 | 0.8864 | 28.99 | 0.9429 |
| 0.8 | 24.13 | 0.8505 | 27.40 | 0.9172 |
| 0.85 | 22.76 | 0.7859 | 25.40 | 0.8600 |
| 0.9 | 21.38 | 0.6890 | 23.50 | 0.7649 |
| 0.95 | 20.10 | 0.5625 | 21.82 | 0.6325 |
| 1.0 | 18.94 | 0.4171 | 20.37 | 0.4735 |




(a) PSNR across turbulence strengths.

(b) SSIM across turbulence strengths.

Figure 6: Generalization performance of the 7-frame U-Net model trained on 0.75 turbulence strength across different turbulence strengths and input settings.

The peak performance at 0.70 test-time turbulence strength is unexpected, showing an equilibrium between remembering the degradation distribution and easiness of restoration. For lower turbulence strength, the model may be overfitting to the stronger degradation seen in training, while for higher turbulence strength the restoration task becomes more difficult and the model's learned features may not be sufficient to recover the clean image. This non-monotonic behavior suggests that the model's generalization is influenced by a complex interplay between the training degradation distribution and the test-time degradation, which could be an interesting direction for further investigation.

### 3.2 Qualitative results


Figure 7: Representative restoration examples from the validation set (0.50 training time turbulence strength). From left to right in each row: degraded input, clean target, and U-Net prediction.

Figure 7 shows representative restoration examples from the validation set. Each row corresponds to one sample, with the degraded input on the left, the clean target in the middle, and the U-Net prediction on the right. The visualized examples are selected to cover a range of scene types (e.g., roads, buildings, parking areas, wild areas) and to highlight typical restoration improvements. The restoration performance of default 0.50 turbulence strength is nearly perfect.

Figure 8 shows representative restoration examples from the validation set with 0.75 training time turbulence strength. Compared to the generalization results in Figure 5, the restoration resolution is relatively high for 0.75 turbulence strength. One difference seen by naked eye is that the color saturation of the restored images are slightly lower than those of the ground truth.

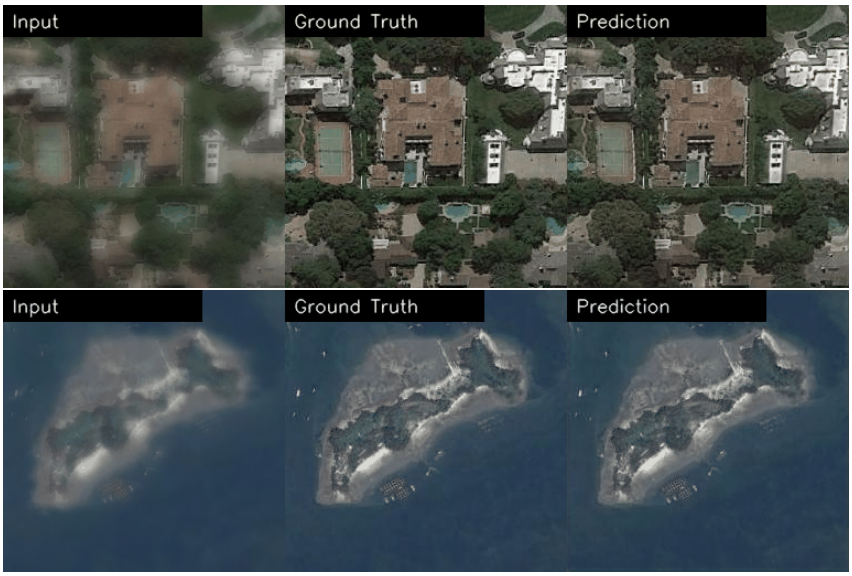



Figure 8: Representative restoration examples from the validation set with 0.75 training time turbulence strength. From left to right in each row: degraded input, clean target, and U-Net prediction.

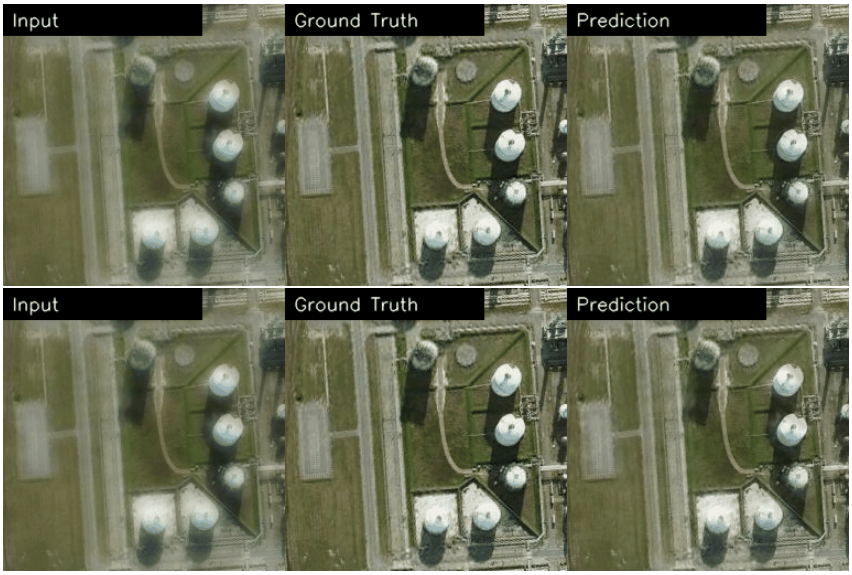
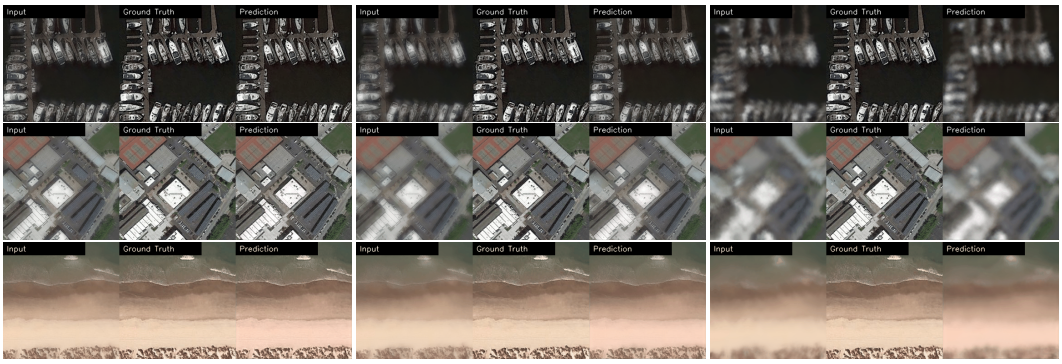



Figure 9: Comparison between U-Net models trained on 7-frame input (top) and 1-frame input (down) with 0.50 training time and 0.60 test time turbulence strength.

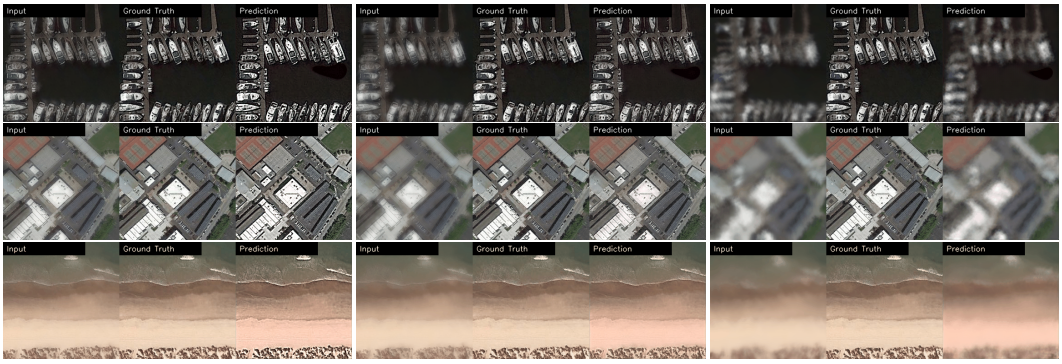
Figure 9 shows the restoration results of the 7-frame U-Net model when test-time input is one-frame and seven-frame with turbulence strength 0.60. The results demonstrate some slight differences, e.g. the white building roof is more clearly restored in the seven-frame input case.

Figure 10 shows the comparison between different turbulence strengths (0.50, 0.75, 1.0) with 7-frame input. The results show that the restoration performance drops significantly as turbulence strength increases.

- For the 7-frame U-Net model trained on 0.50 turbulence strength, the restoration performance drops significantly as turbulence strength increases. The restored images with 0.75 turbulence strength still have some blur in fine details, while the restored images with 1.0 turbulence strength are severely blurred and distorted.
- For the 7-frame U-Net model trained on 0.75 turbulence strength, the problem of restoration performance at 0.50 test-time turbulence strength is that the constrast of the images are too high; while for 1.0 test-time turbulence strength, the restored images are still blurred. Moreover, we spotted a black patch in the restored images.




(a) Turbulence strength 0.50. (b) Turbulence strength 0.75. (c) Turbulence strength 1.0.




(d) Turbulence strength 0.50. (e) Turbulence strength 0.75. (f) Turbulence strength 1.0.

Figure 10: Comparison of restoration results across different turbulence strengths (0.50, 0.75, 1.0) with 7-frame input. Figure (a), (b), (c) are the results of the 7-frame U-Net model trained on 0.50 turbulence strength, while Figure (d), (e), (f) are the results of the 7-frame U-Net model trained on 0.75 turbulence strength.

### 3.3 Discussion

The results point to several insights beyond the raw validation scores. First, temporal stacking is consistently useful. Under the 0.50 training-time turbulence setting, the 7-frame U-Net reaches 35.341 dB PSNR and 0.9804 SSIM, compared with 31.483 dB and 0.9581 SSIM for the corresponding 1-frame model. Under the stronger 0.75 setting, the 7-frame model also outperforms the 1-frame model, with 31.783 dB and 0.9557 SSIM versus 29.463 dB and 0.9256 SSIM. The improvement is therefore not restricted to an easy degradation regime. The model benefits from multiple degraded observations because different frames preserve different local structures after random warping and

blur. The qualitative comparisons support this interpretation: roads, rooftops, parking boundaries, and building edges are sharper when seven frames are available.

Second, the new 0.75-strength experiments show that training turbulence strength behaves like a degradation-domain prior rather than a simple robustness knob. The model trained at 0.50 performs best when the test degradation is also mild, and its generalization curves decrease almost monotonically as turbulence strength increases. In contrast, the model trained at 0.75 does not peak at the easiest test setting. Its best performance appears around 0.70 test-time turbulence strength for both 1-frame and 7-frame inputs. This non-monotonic pattern suggests a mismatch effect: a model trained mainly to undo stronger blur and warping can over-correct or distort images from milder turbulence, while still struggling once the test corruption becomes more severe than the training distribution. The visual comparison at 0.50 test-time strength, where the 0.75-trained model tends to produce overly high contrast, is consistent with this explanation.

Third, the 7-frame advantage remains visible across both generalization studies, but the size of the advantage depends on the degradation regime. For the 0.50-trained model, seven frames are clearly better than one frame at every tested strength, although the absolute performance drops sharply as turbulence approaches 1.0. For the 0.75-trained model, the multi-frame gain becomes especially important near the moderate-strength region where the model is best matched to the test distribution: at 0.70, the 7-frame result exceeds the 1-frame result by more than 3 dB PSNR. This indicates that temporal redundancy is most useful when the input sequence still contains recoverable but differently corrupted evidence. When turbulence becomes too strong, the sequence supplies more observations, but those observations may all have lost the same fine-scale information.

Finally, the experiments clarify the limits of the current baseline. A single U-Net trained at one turbulence strength does not provide uniform restoration quality across all turbulence levels. The 0.50 model gives excellent in-distribution restoration, but its performance degrades rapidly under stronger turbulence. The 0.75 model is better matched to moderate degradation, but it sacrifices mild-case performance and still leaves blur or artifacts at 1.0 strength. These findings suggest that future models should either train on a broader turbulence-strength distribution or explicitly condition the network on degradation parameters such as estimated strength, wind, or blur severity. The current results are therefore best interpreted as a strong simulator-domain baseline and an informative ablation of temporal input and degradation mismatch, rather than as evidence of complete robustness to real atmospheric turbulence.

## 4 Conclusion

This project studies adaptive atmospheric turbulence restoration for remote sensing images through a compact but physically grounded learning pipeline. We combine clean remote sensing imagery from UC Merced and NWPU-RESISC45, a TurbulenceSim-style simulator for generating paired turbulence sequences, and U-Net restoration models trained with an $L_1$ objective. The best 0.50-strength 7-frame model achieves 35.34 dB PSNR and 0.9804 SSIM on the validation split, while the stronger 0.75-strength setting still reaches 31.78 dB PSNR and 0.9557 SSIM with seven-frame input. Across both settings, multi-frame restoration consistently outperforms the corresponding single-frame baseline.

The main contribution of the project is a reproducible remote-sensing turbulence restoration pipeline that connects physically motivated synthetic degradation, sequence-based input aggregation, and quantitative evaluation across turbulence strengths. The new generalization experiments show that temporal information is valuable, but also that degradation mismatch matters: a model trained on mild turbulence performs very well in distribution but degrades under stronger corruption, while a model trained on 0.75 turbulence is better tuned to moderate degradation and can underperform on milder inputs because of over-restoration effects.

These findings point to several next steps. A stronger system should train on mixed or continuous turbulence strengths, condition the restoration network on degradation parameters, and compare U-Net results against the GAN and diffusion models already scaffolded in the repository. Broader data coverage and real turbulence evaluation are also necessary to measure the simulation-to-reality gap. Overall, the current work provides a strong baseline and a clearer empirical lesson: multi-frame evidence improves restoration, but robust adaptive optics restoration also requires explicit handling of turbulence-strength variation.

## References


Nicholas Chimitt and Stanley H. Chan. Simulating anisoplanatic turbulence by sampling inter-modal and spatially correlated Zernike coefficients. *arXiv preprint arXiv:2004.11210*, 2020.

Paul Hill, Nantheera Anantrasirichai, Alin Achim, and David Bull. Deep learning techniques for atmospheric turbulence removal: A review. *arXiv preprint arXiv:2409.14587*, 2024.

Anshuman Jaiswal, Zhangyang Wang, and Stanley H. Chan. Physics-driven turbulence image restoration with stochastic refinement. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023.

Kaixuan Jin, Yuhong Guo, Changshui Zhang, and Stanley H. Chan. Neutralizing the impact of atmospheric turbulence on complex scene imaging via deep learning. *Nature Machine Intelligence*, 3(10):876–884, 2021.

Kartikeya Mei and Vishal M. Patel. LTT-GAN: Looking through turbulence by inverting GANs. *IEEE Journal of Selected Topics in Signal Processing*, 17(1):150–163, 2023.

Yuan Wang, Haoyi Zhang, and Hongkai Xiong. Zero-shot image restoration using denoising diffusion null-space model. In *International Conference on Learning Representations*, 2023.

Zhou Wang, Alan C. Bovik, Hamid R. Sheikh, and Eero P. Simoncelli. Image quality assessment: from error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13(4):600–612, 2004.